

Discussion 2 Linux Kernel Mailing List – Sheaves

Discussion Topic:

Linux is an open-source operating system, which means that the source code can be downloaded, modified, compiled locally, and run on your own computer with your custom modifications. These modifications could be contributed to the community if they are determined to be valid improvements.

To see what improvements are being made in real time, go to the Linux Kernel Mailing List archive. Scroll to the bottom of the page and select "read yesterday's message" (today's message may be empty, depending on your time zone.) Browse through the list to see what is being discussed. Select one of the message threads and explain the following:

What is Linux currently doing?

What is the improvement that is being proposed?

Why is that an improvement? (Or explain why it is not.)

Include a link to the message thread you select in your initial post.

selected message <https://lkml.org/lkml/2026/5/16/14> :

...

From "Harry Yoo (Oracle)" <>
Subject [PATCH RFC 0/8] mm/slab: enable runtime sheaves tuning
Date Sat, 16 May 2026 01:24:24 +0900

Background
=====

Sheaves were introduced in v6.18, and starting from v7.0, they are enabled for all slab caches (except for `kmem_cache{,_node}`). In the pre-sheaves era, there was a `cpu_partial` parameter to tune the number of objects cached per CPU. However, sheaves don't have an equivalent and the sheaf capacity is determined in the kernel code.

The goal is to allow tuning sheaves at runtime by the next LTS.

Overview
=====

This patchset does two main things:

1. Make the `sheaf_capacity` sysfs attribute writable so that the number of objects cached per CPU can be changed at runtime, and
2. Expose `MAX_FULL_SHEAVES` and `MAX_EMPTY_SHEAVES` as sysfs attributes rather than constants, so that users can tune them.

Measuring the performance impact of these tunables is TBD.

Roughly, the sequence to change sheaf_capacity is as follows:

1. Disable sheaves. Make all online CPUs replace their main sheaves with the bootstrap sheaf under local_lock and wait for completion.
2. Wait for all in-flight RCU callbacks to be processed.
3. Flush and free all existing sheaves.
4. Re-enable sheaves with a new capacity.

Challenges

=====

1. Allocations and frees can happen concurrently at any point between these steps, and we cannot introduce heavyweight synchronization mechanisms on the fastpath.
2. Currently, cache_has_sheaves() checks whether a cache has sheaves. This works now because sheaves cannot be enabled or disabled once the cache is created.

The question "Does this cache has sheaves?" should be split into "Does this cache support sheaves?" and "Does this CPU actually has sheaves enabled right now?".

3. Once the sheaf capacity update is complete, no sheaf with stale capacity must remain. Flushing and freeing all existing sheaves is relatively simple, but under the current design it is quite challenging to prevent sheaves with stale capacity to be installed to the pcs or the barn. Reading s->sheaf_capacity without an expensive synchronization primitive is racy.

Patch 6 introduces a copy of s->sheaf_capacity to struct slab_percpu_sheaves to address this. pcs->capacity is copied from s->sheaf_capacity and it is stable under local_lock. If s->sheaf_capacity and pcs->capacity don't match, the sheaf_capacity writer is responsible for flushing and freeing them before completing the process.

Patch Sequence

=====

Patch 1-3: A per-sheaf capacity is required for the following steps, but I didn't want to grow struct slab_sheaf. So patch 1 drops the cache pointer (which was used only on the slowpath), patch 2 changes sheaf_capacity from unsigned int to unsigned short, and patch 3 adds per-sheaf capacity.

Actually, the size is shrunken after those patches.

After (24 bytes, excluding the objects flex array):

```
struct slab_sheaf {
    union {
        struct rcu_head rcu_head;
        struct list_head barn_list;
    };
};
```

```

        bool pfmemalloc;
    };
    unsigned short capacity;
    unsigned short size;
    int node;
    void *objects[];
};

```

Patch 4 allows bootstrap_cache_sheaves() to fail so that it can be used to re-enable sheaves without panicking the kernel.

Patch 5 splits cache_has_sheaves() into cache_supports_sheaves() and pcs_has_sheaves().

Patch 6 enables tuning the sheaf capacity at runtime.

Patch 7 adds lockdep asserts to verify the new rule "Always hold local_lock when accessing the barn" to make sure there is no sheaf with stale capacity.

Patch 8 turns MAX_FULL_SHEAVES and MAX_EMPTY_SHEAVES into sysfs attributes (max_full_sheaves, max_empty_sheaves) and allows tuning.

RFC V1 is also available in git at:

<https://git.kernel.org/pub/scm/linux/kernel/git/harry/linux.git/log/?h=sheaves-tuning-rfc-v1r1>

Signed-off-by: Harry Yoo (Oracle) <harry@kernel.org>

Harry Yoo (Oracle) (8):

```

mm/slab: do not store cache pointer in struct slab_sheaf
mm/slab: change sheaf_capacity type to unsigned short
mm/slab: track capacity per sheaf
mm/slab: allow bootstrap_cache_sheaves() to fail
mm/slab: rework cache_has_sheaves() to check immutable properties only
mm/slab: allow changing sheaf_capacity at runtime
mm/slab: add pcs->lock lockdep assert when accessing the barn
mm/slab: allow changing max_{full,empty}_sheaves at runtime

```

```

include/linux/slab.h      |    8 +-
mm/slab.h                 |   40 ++-
mm/slab_common.c          |    2 +-
mm/slub.c                 |  715 ++++++-----

```

--

```

tools/include/linux/slab.h |   14 +-
tools/testing/shared/linux.c |    4 +-
6 files changed, 563 insertions(+), 220 deletions(-)

```

base-commit: e98d21c170b01ddef366f023bbfcf6b31509fa83

change-id: 20260515-sheaves-tuning-elf897dc7f5e

Best regards,

--

Cheers,

Harry / Hyeonggon

My Post:

Hello Class

This was an interesting assignment for me; it shows how open source projects handle [...]. Anyhow, I chose the thread “[PATCH RFC 0/8] mm/slab: enable runtime sheaves tuning” by Harry Yoo from Oracle (<https://lkml.org/lkml/2026/5/16/14>).

The thread is about improving the runtimes of the sheaves. Before diving into the path proposal, it is important to define what a slab, slab allocator, and sheaf. I personally had to do some research as I was not familiar with what a sheaf even is.

A slab is a memory-management data structure that the Linux kernel uses to store multiple fixed-size objects. In this context, a “fixed-size object” is a reusable piece of kernel memory. Kernel memory is the part of a computer’s memory that is used by the OS kernel itself. The original Linux slab allocator cached objects in a process called object caching. Object caching reduces allocation cost by keeping frequently used kernel objects in cache memory, allowing them to be quickly available for reuse instead of repeatedly creating and destroying them (Bonwick, 1994).

A slab allocator is the kernel memory-management system that manages these slabs and stores kernel memory objects. In the context of Linux, the Linux kernel’s slab allocator allocates small, usually sub-page, that is chunks of memory, speeding up object allocation and freeing, which is important for overall system performance (Corbet, 2025)

A sheaf is a newer caching process; it is actually a new caching layer that the slab allocator uses to cache small kernel memory objects. Instead of requiring a CPU to interact with shared allocator structures every time it allocates or frees a kernel memory object, the sheaf layer allows many of these operations to happen locally on that CPU, more specifically, in the L1/L2/L3 cache hardware CPU cache memory. This can improve significantly the performance of a system (Corbet, 2025).

In the context of the PATCH RFC 0/8 thread, Linux is currently improving its memory-management system, specifically the slab allocator. The Yoo patch proposal focuses on making sheaves more flexible after the system is already running, that is, at runtime. Sheaves were introduced in v6.18, and starting from v7.0, they are enabled for all slab caches (except for `kmem_cache{, _node}``). In other words, their holding capacity is currently determined by kernel code and cannot be adjusted at runtime, meaning that their capacity behavior is static rather than dynamic, not providing administrators and developers with a simple way to tune how many objects are cached per CPU after the cache has been created.

To improve the sheave capacity behavior, Yoo proposes to allow the sheave capacity to be tunable at runtime, by making the `sheaf_capacity sysfs`` attribute writable so the number of objects cached per CPU can be changed at runtime. He also proposes that the `MAX_FULL_SHEAVES`` and `MAX_EMPTY_SHEAVES`` be `sysfs`` attributes, and renaming them `max_full_sheaves`` and `max_empty_sheaves``, turning them from constant/static to dynamic runtime tunables so users can tune how many full and empty sheaves can be kept.

This would improve systems and workloads that may need different memory-allocation behavior depending on their workload patterns, resources, and performance requirements. For example, a server that requires many small kernel allocations may benefit from different sheaf capacity settings; on the other hand, a desktop system may not need the same caching behavior, and the sheaf capacity does need to be smaller or tuned differently to avoid using extra memory for little performance benefit. Overall, sheave

capacity runtime tuning would be a significant improvement, allowing administrators and kernel developers to try out different capacity values right at runtime without having to rebuild the kernel or hardcode one behavior for all systems. In other words, the proposed change makes the slab allocator more adaptable.

-Alex

References:

Bonwick, J. (1994). *The slab allocator: An object-caching kernel memory allocator* [PDF]. Sun Microsystems. <https://web.eecs.umich.edu/~ryanph/jhu/cs318/fall18/readings/slab.pdf>

Corbet, J. (2025, February 24). *Slabs, sheaves, and barns*. LWN.net. <https://lwn.net/Articles/1010667/>